
API Security Risk Analysis Report

API Security Risk Analysis

JSONPlaceholder API

Date: 20 June 2026

*Intern: **PATIL TEJASWINI ANIL***

CIN ID: FIT/MAY26/CS8743

Organization: Future Interns

Task: Cyber Security Task 3 – API Security Risk Analysis

Table of Contents

1. Executive Summary	1
2. Disclaimer	2
3. Tools Used	3
4. Scope of Assessment	4
5. Methodology	5
6. API Overview	6
7. Authentication Assessment	7
8. Access Control Assessment	8
9. Risk Assessment Summary	9
10. Finding 1 – Open Endpoint Access	10
11. Finding 2 – Missing Authentication Controls	11
12. Finding 3 – Missing Rate Limiting Indicators	12
13. Finding 4 – Excessive Data Exposure	13
14. Screenshots & Evidence	14
15. Recommendations	15
16. Conclusion	16
17. References	17

Disclaimer

This API Security Risk Analysis Report has been prepared as part of the Future Interns Cyber Security Internship Program.

The purpose of this assessment is to perform a read-only security review of a public testing API and identify potential security risks from an API security perspective.

No exploitation, bypass attempts, brute-force attacks, denial-of-service testing, or unauthorized activities were performed during this assessment.

Testing was limited to documentation review, API endpoint inspection, response analysis, and HTTP header examination.

Access Control Assessment

Access Control Assessment

The tested endpoints were publicly accessible and did not require authentication or authorization checks. Since JSONPlaceholder is a public testing API, this behavior is expected. However, in production APIs, access control should ensure that users can only access resources they are authorized to view.

Input Validation Note

Input validation testing was not actively performed because the assessment followed a read-only and non-exploitative scope. No malicious payloads or bypass attempts were used.

Tools Used

Tool	Purpose
Postman	API Request Testing
Google Chrome	Documentation Review
JSONPlaceholder API	Target API
Microsoft Word	Report Preparation

Scope

API	Description
https://jsonplaceholder.typicode.com	Public API for testing and learning

Assessment Scope:

- GET Requests Only
- Documentation Review
- Header Inspection
- Response Analysis
- Authentication Review

Out of Scope:

- Exploitation
- DoS Testing
- Brute Force
- Authentication Bypass
- Data Modification

Risk Assessment

Finding	Description	Severity
Open Endpoint Access	API endpoints accessible without authentication	Medium
Missing Authentication Controls	No visible token or authentication mechanism	Medium
Missing Rate Limiting Indicators	No visible rate-limiting headers observed	Medium
Excessive Data Exposure	User details exposed in API response	Low

API Overview

API Tested:

<https://jsonplaceholder.typicode.com>

Endpoints Reviewed:

1. /users
2. /posts

Method Used:

GET Requests

Response Format:

JSON

Methodology

1. Selected JSONPlaceholder public API.
2. Reviewed API documentation.
3. Performed read-only GET request testing using Postman.
4. Analyzed JSON responses.
5. Reviewed HTTP response headers.
6. Assessed authentication and access control mechanisms.
7. Identified potential security risks.
8. Assigned risk severity levels.
9. Provided remediation recommendations.

Finding 1 – Open Endpoint Access

Description

The /users endpoint was accessible without authentication.

The API returned user information directly without requiring login credentials, API keys, or access tokens.

Evidence

Endpoint:

<https://jsonplaceholder.typicode.com/users>

Method:

GET

Status:

200 OK

Impact

If similar behavior exists in a production API, unauthorized users may access data without verification.

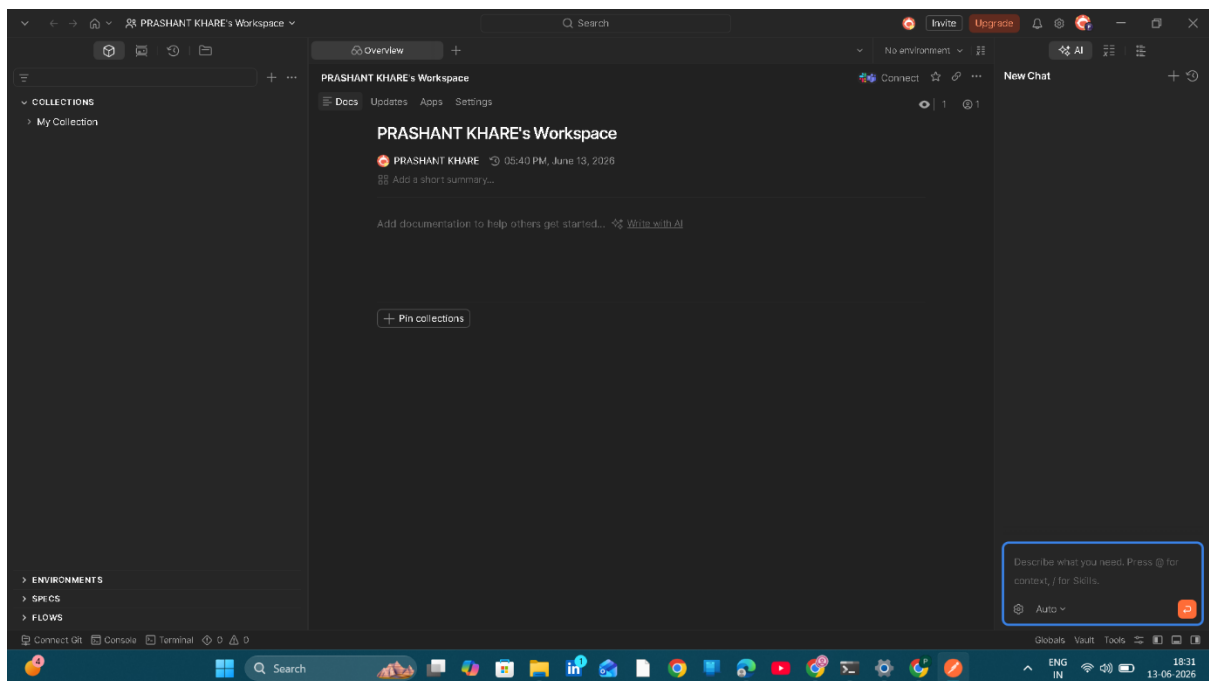
Severity

Medium

Recommendation

- Implement authentication mechanisms.
- Require API keys or JWT tokens.

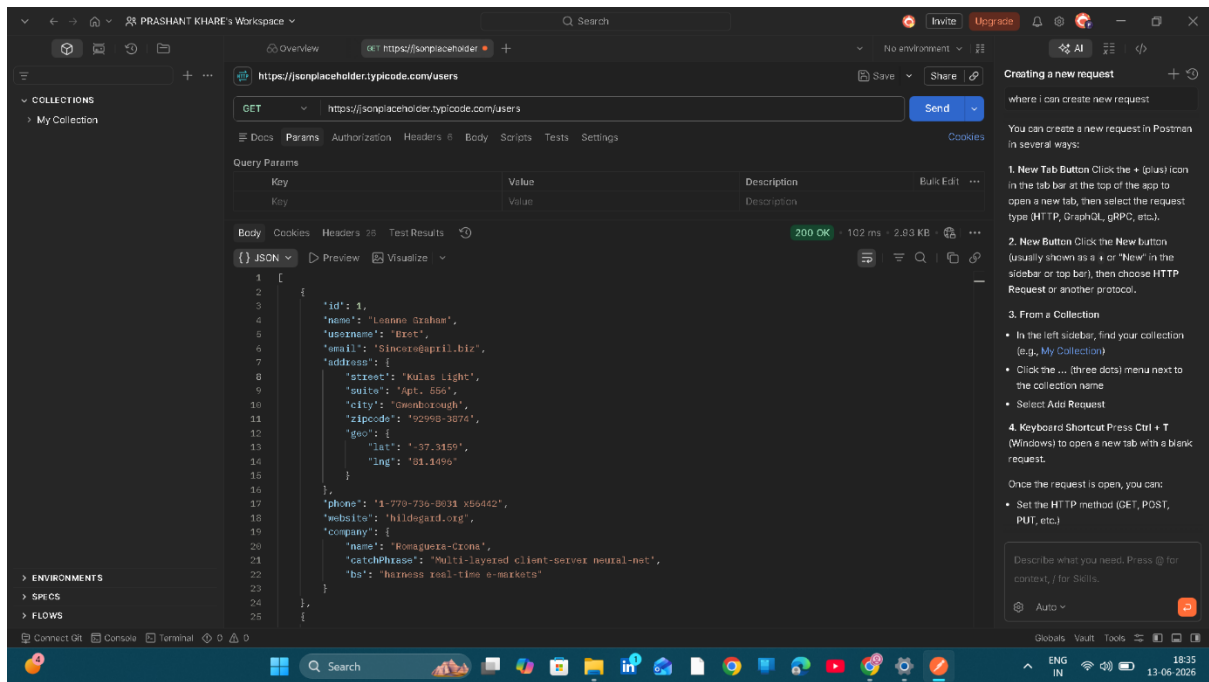
- Restrict sensitive endpoints.



1.1 Postman Workspace Setup

Description:

The Postman application was successfully installed and configured for API testing. A dedicated workspace was created to perform read-only API security analysis activities. Postman was used to send HTTP requests, inspect API responses, and review HTTP headers.



1.2 GET /users Response to /users Endpoint

Description:

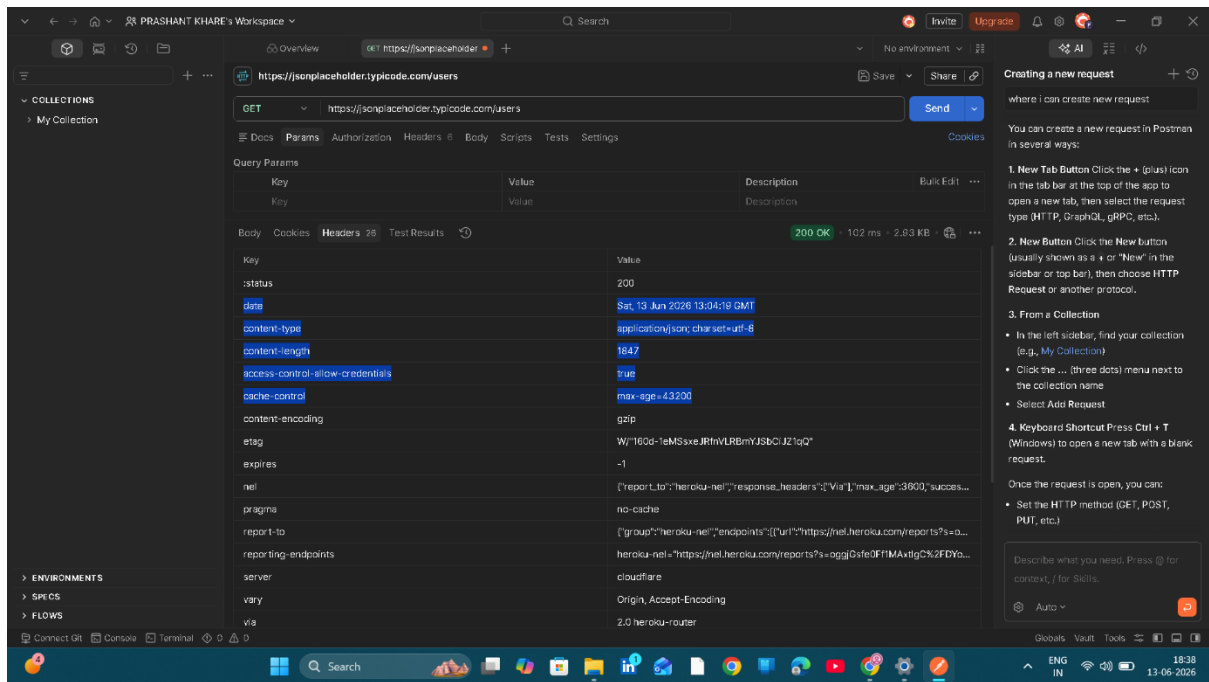
A GET request was sent to the /users endpoint of the JSONPlaceholder API. The server responded with HTTP Status Code 200 OK and returned user information in JSON format. The response included user details such as ID, name, username, email address, phone number, website, company information, and address details.

Security Observation:

The endpoint was accessible without authentication, API keys, or access tokens.

Risk Level:

Medium



1.3 Headers Analysis

Description:

Response headers were inspected to identify security-related configurations. The API returned standard HTTP headers including Content-Type, Cache-Control, Server, Date, and Content-Encoding.

Observed Headers:

```
Content-Type: application/json
Cache-Control: max-age=43200
Server: cloudflare
Content-Encoding: gzip
```

Security Observation:

No visible rate-limiting headers such as:

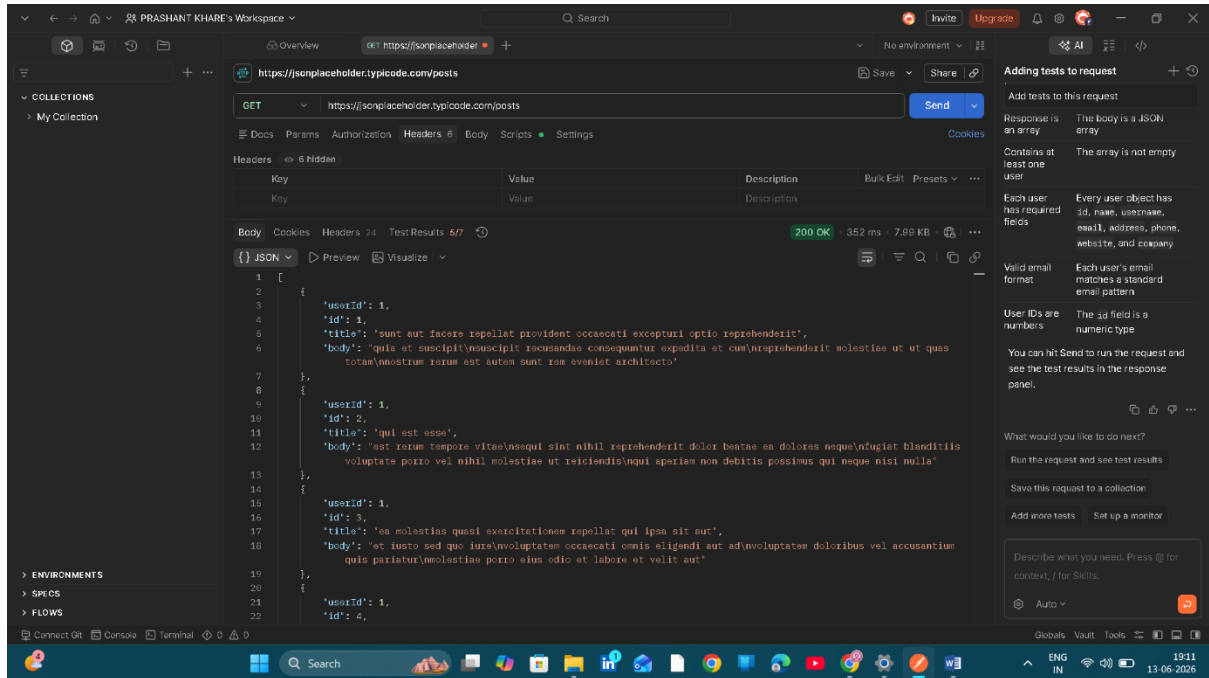
```
X-RateLimit-Limit
X-RateLimit-Remaining
```

were observed during testing.

Risk Level:

Medium

Detailed Analysis of /posts Endpoint



2.1 Detailed Analysis of /posts Endpoint

Description:

A GET request was sent to the `/posts` endpoint of the JSONPlaceholder API using Postman. The API successfully returned HTTP Status Code **200 OK** and provided post-related information in JSON format.

The response contained:

```
userId
id
title
body
```

The endpoint returned multiple records without requiring authentication, API keys, or access tokens.

Security Observation

The API allows public access to post data through a simple GET request.

No authentication or authorization checks were observed during testing.

Business Impact

If similar behavior exists in production APIs containing sensitive information, unauthorized users may access data without proper verification.

Severity

☐ Medium

Recommendation

- Implement authentication mechanisms.
- Validate user authorization before returning data.
- Restrict access to sensitive resources.
- Apply role-based access control (RBAC).

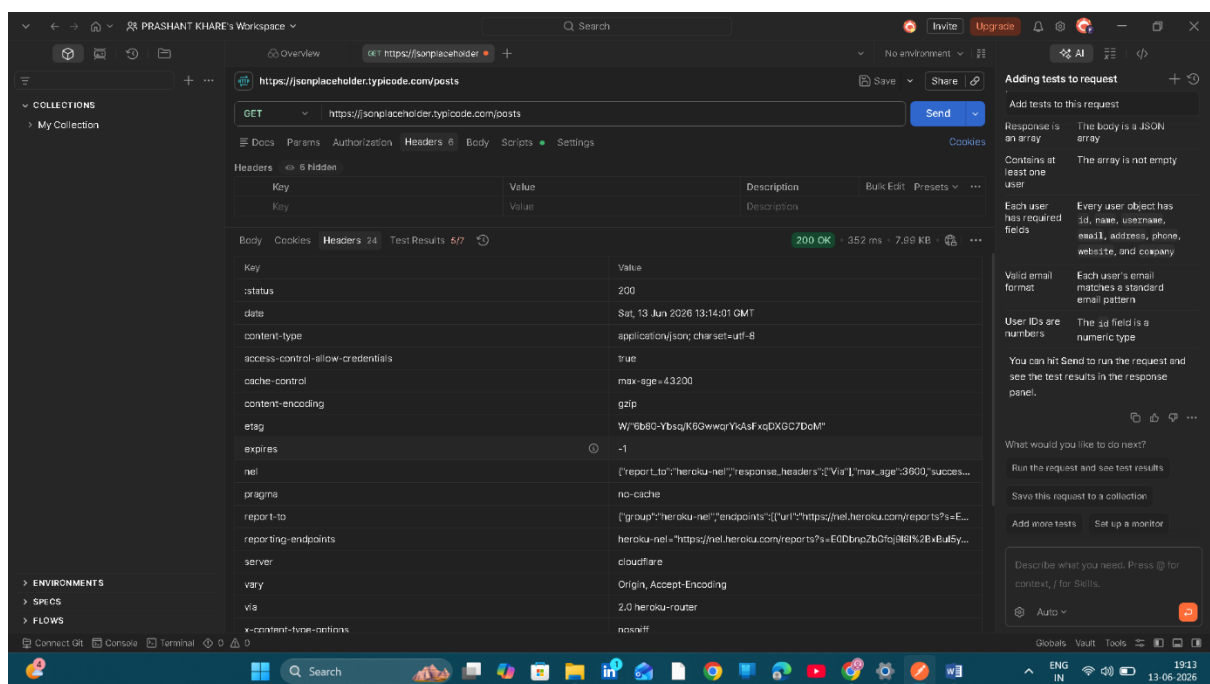


Fig 2.2 Header Security Analysis for /posts Endpoint

Description:

The HTTP response headers for the `/posts` endpoint were analyzed using Postman. The API returned a successful **200 OK** response and several standard security and operational headers.

Observed Headers:

```
Content-Type: application/json
Cache-Control: max-age=43200
Content-Encoding: gzip
Server: cloudflare
X-Content-Type-Options: nosniff
```

Security Assessment

The API implements some basic security headers such as:

- X-Content-Type-Options: nosniff
- Content-Type
- Cache-Control

These headers help improve response handling and reduce certain client-side security risks.

Missing Controls Observed

No visible authentication-related headers were observed:

```
Authorization
Bearer Token
API Key
```

No visible rate-limiting headers were observed:

```
X-RateLimit-Limit
X-RateLimit-Remaining
X-RateLimit-Reset
```

Business Impact

If production APIs do not implement proper authentication and rate-limiting controls, attackers may abuse publicly accessible endpoints, consume excessive resources, or access sensitive information.

Severity

☐ Medium

Recommendation

- Implement API authentication mechanisms.
- Apply rate limiting and request throttling.
- Continuously monitor API traffic.
- Restrict access to sensitive resources.

Conclusion

This API Security Risk Analysis was conducted on the JSONPlaceholder public testing API using a read-only assessment approach. The evaluation focused on API endpoint accessibility, authentication mechanisms, access control, HTTP response headers, and response data exposure.

During the assessment, several security observations were identified, including publicly accessible endpoints, lack of visible authentication controls, absence of rate-limiting indicators, and exposure of multiple user-related data fields in API responses. While these behaviors are expected in a public testing API designed for learning and development purposes, similar conditions in production environments could increase the risk of unauthorized access, information disclosure, and API abuse.

The assessment demonstrated the importance of implementing strong authentication, proper authorization controls, rate limiting, and data minimization practices to improve API security posture. Organizations should regularly review and monitor their APIs to ensure that only authorized users can access sensitive resources and that unnecessary data exposure is minimized.

Overall, this assessment provided practical experience in analyzing APIs from a security consultant's perspective, identifying potential risks, evaluating business impact, and recommending appropriate remediation measures without performing any exploitation or unauthorized activities. The exercise highlights the critical role of API security in modern SaaS applications and reinforces the importance of secure API design and governance.

References

1. JSONPlaceholder – Fake Online REST API for Testing and Prototyping
<https://jsonplaceholder.typicode.com>
2. Postman API Platform Documentation
<https://learning.postman.com>
3. OWASP API Security Top 10 Project
<https://owasp.org/www-project-api-security>
4. OWASP API Security GitHub Repository
<https://github.com/OWASP/API-Security>
5. API Security Checklist (Shieldfy)
<https://github.com/shieldfy/API-Security-Checklist>
6. Public APIs Collection Repository
<https://github.com/public-apis/public-apis>
7. Future Interns – Cyber Security Task 3 Guidelines
API Security Risk Analysis (2026)
8. HTTP Response Header Reference
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers>
9. REST API Security Best Practices
<https://owasp.org/www-project-api-security>